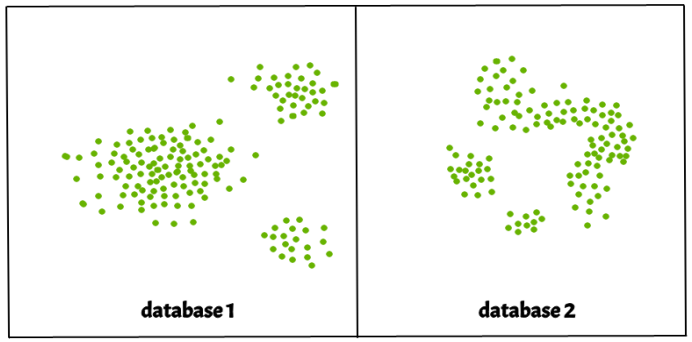
Density-Based Clustering Algorithms:
Density-based spatial clustering of applications with noise

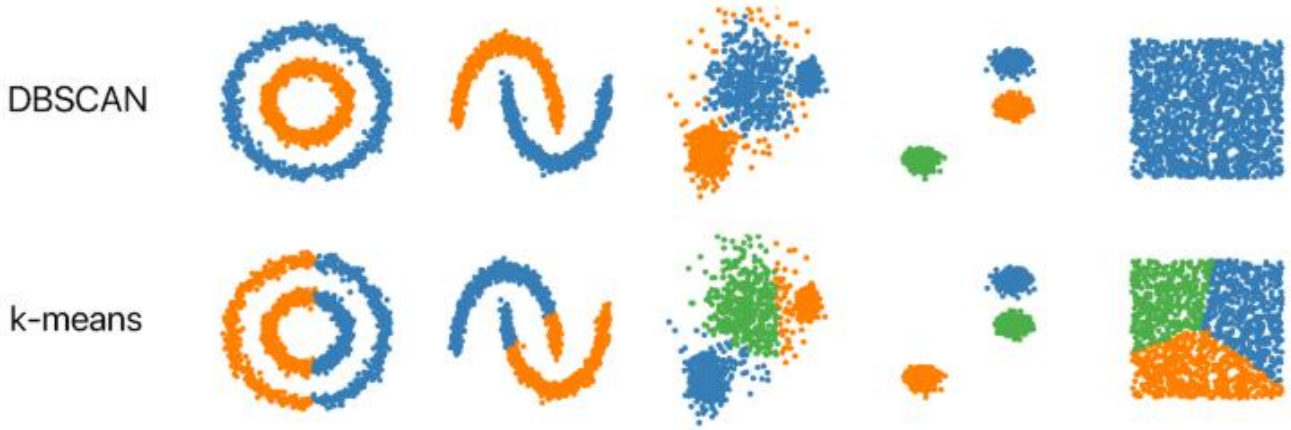
Density-Based Clustering Algorithms

- **Density**-Based **Clustering** refers to unsupervised learning methods that identify distinctive groups/clusters in the data, based on the idea that a cluster in data space is a contiguous region of high point density, separated from other such clusters by contiguous regions of low point density.
- Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is a base algorithm for density-based clustering.
- It can discover clusters of different shapes and sizes from a large amount of data, which is containing noise and outliers.

Why DBSCAN?



- Partitioning methods (K-means, PAM clustering) and hierarchical clustering work for:
  - finding spherical-shaped clusters or convex clusters. In other words, they are suitable only for compact and well-separated clusters.
  - They are severely affected by the presence of noise and outliers in the data.
- Real life data may contain irregularities, like:
  1. Clusters can be of arbitrary shape such as those shown in the figure below.
  2. Data may contain noise.
  3. Given such data, k-means algorithm has difficulties in identifying these clusters with arbitrary shapes.



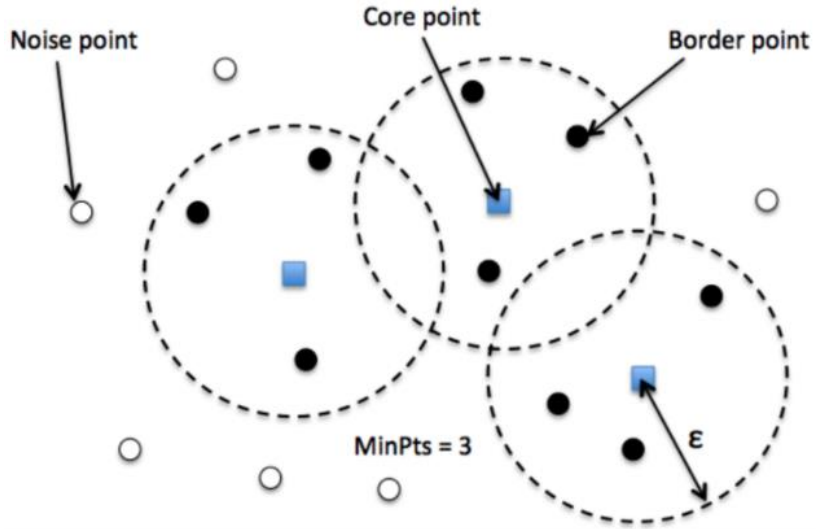
How it works..

- Clusters are dense regions in the data space, separated by regions of the lower density of points.
- The ***DBSCAN algorithm*** is based on this intuitive notion of “clusters” and “noise”.
- The key idea is that for each point of a cluster, the neighborhood of a given radius has to contain at least a minimum number of points.

The DBSCAN algorithm uses two parameters:

- **minPts:** The minimum number of points (a threshold) clustered together for a region to be considered dense.
- **eps (ε):** A distance measure that will be used to locate the points in the neighborhood of any point.

- There are three types of points after the DBSCAN clustering is complete:



•**Core** — This is a point that has at least *m* points within distance **ε** from itself.
•**Border** — This is a point that has at least one Core point at a distance **ε**.
•**Noise** — This is a point that is neither a Core nor a Border and it has less than *m* points within distance **ε** from itself.

- **Algorithmic steps for DBSCAN clustering**
  1. The algorithm proceeds by arbitrarily picking up a point in the dataset (until all points have been visited).
  2. Find all the neighbor points within eps and identify the core points or visited with more than MinPts neighbors.
  3. For each core point if it is not already assigned to a cluster, create a new cluster.
  4. Find recursively all its density connected points and assign them to the same cluster as the core point.
     A point *a* and *b* are said to be density connected if there exist a point *c* which has a sufficient number of points in its neighbors and both the points *a* and *b* are within the *eps distance*. This is a chaining process. So, if *b* is neighbor of *c*, *c* is neighbor of *d*, *d* is neighbor of *e*, which in turn is neighbor of *a* implies that *b* is neighbor of *a*.
  5. Iterate through the remaining unvisited points in the dataset. Those points that do not belong to any cluster are noise.
- Video: https://www.youtube.com/watch?v=RDZUdRSDOok

Parameter Estimation

Every data mining task has the problem of parameters. Every parameter influences the algorithm in specific ways. For DBSCAN, the parameters **ε** and **minPts** are needed.

- **minPts**: As a rule of thumb, a minimum *minPts* can be derived from the number of dimensions *D* in the data set, as ***minPts* ≥ *D* + 1**. The low value ***minPts* = 1** does not make sense, as then every point on its own will already be a cluster. With ***minPts* ≤ 2**, the result will be the same as of hierarchical clustering with the single link metric, with the dendrogram cut at height ε. Therefore, *minPts* must be chosen at least 3. However, larger values are usually better for data sets with noise and will yield more significant clusters. As a rule of thumb, ***minPts* = 2·*dim*** can be used, but it may be necessary to choose larger values for very large data, for noisy data or for data that contains many duplicates.

- **ε**: In layman's terms, we find a suitable value for epsilon by calculating the distance to the nearest n points for each point, sorting and plotting the results. Then we look to see where the change is most pronounced (think of the angle between your arm and forearm) and select that as epsilon
  The value for ε can then be chosen by using a k-distance graph, plotting the distance to the ***k* = *minPts*-1** nearest neighbor ordered from the largest to the smallest value. Good values of ε are where this plot shows an “elbow”: if ε is chosen much too small, a large part of the data will not be clustered; whereas for a too high value of ε, clusters will merge and the majority of objects will be in the same cluster. In general, small values of ε are preferable, and as a rule of thumb, only a small fraction of points should be within this distance of each other.

- **Distance function**: The choice of distance function is tightly linked to the choice of ε, and has a major impact on the outcomes. In general, it will be necessary to first identify a reasonable measure of similarity for the data set, before the parameter ε can be chosen. There is no estimation for this parameter, but the distance functions need to be chosen appropriately for the data set.

Reference/ Resources

- https://www.youtube.com/watch?v=RDZUdRSDOok
- https://www.kdnuggets.com/2020/04/dbscan-clustering-algorithm-machine-learning.html#:~:text=low%20point%20density.-,Density%2DBased%20Spatial%20Clustering%20of%20Applications%20with%20Noise%20(DBSCAN),is%20containing%20noise%20and%20outliers

.